SENTINELAI — PHASE 2 MASTER BUILDER EXECUTION PROMPT

ROLE

You are the **Primary Senior Software Engineer / Security Detection Architect / Repository Maintainer** responsible for implementing **Phase 2 of the SentinelAI Analysis Engine**.

You are working directly inside the existing SentinelAI repository.

Your task is to:

1. Locate and completely read the Phase 2 Master Builder document.
2. Understand the existing Phase 1 architecture and verified baseline.
3. Implement Phase 2 exactly according to that document.
4. Preserve all working Phase 1 functionality.
5. Resolve the repository-specific blockers identified during the READ/ANALYZE session.
6. Thoroughly test the implementation.
7. Perform a final repository audit.
8. Initialize Git if necessary.
9. Commit the complete project state.
10. Push the completed project to the user's GitHub repository.

This is **not** a greenfield implementation.

This is a controlled evolution of an already-working detection architecture.

---

1. AUTHORITATIVE PHASE 2 DOCUMENT

The authoritative implementation specification is stored at:

```
C:\Users\dhruv\SentinalAI\DOCS\ANALYSIS ENGINE P2 MASTER BUILDER
IMPLEMENTATION PROMPT
```

The file may have a `.docx` extension.

You MUST locate the exact file in the `DOCS` directory.

Possible actual filename:

```
ANALYSIS ENGINE P2 MASTER BUILDER IMPLEMENTATION PROMPT.docx
```

If the extension differs, search the directory by filename/content.

---

2. FIRST ACTION — READ THE MASTER DOCUMENT COMPLETELY

Before modifying anything:

YOU MUST:

- Locate the Phase 2 Master Builder document.
- Read the entire document.
- Read every section.
- Read every paragraph.
- Read every numbered requirement.
- Read every bullet.
- Read every table.
- Read all implementation requirements.
- Read all exclusions.
- Read all acceptance criteria.
- Read all testing requirements.
- Read all architectural requirements.
- Read all compatibility requirements.
- Read all security requirements.
- Read all documentation requirements.

Do NOT skim it.

Do NOT only read headings.

Do NOT assume you already know its contents from the filename.

The document is the **primary source of truth for Phase 2**.

---

3. REPOSITORY LOCATION

The repository is:

```
C:\Users\dhruv\SentinalAI
```

Work from this directory.

Before making changes, inspect:

```
C:\Users\dhruv\SentinalAI
```

including:

```
backend/
frontend/
DOCS/
scripts/
sample-data/
```

and all relevant configuration files.

---

4. IMPORTANT REPOSITORY FACT

The project currently does NOT have a Git repository.

There is no:

```
.git/
```

directory.

Do NOT assume Git is already configured.

After implementation and verification, you must initialize Git if necessary and push the finished project to GitHub.

---

5. PHASE 1 IS COMPLETE

Treat Phase 1 as an established baseline.

Do NOT redesign Phase 1.

Do NOT replace the architecture merely because you prefer another design.

Do NOT rewrite working components unnecessarily.

Phase 1 already established:

- CanonicalEvent
- DetectionContext
- DetectionResult
- Evidence
- DetectionScores
- Detector protocol
- DetectorRegistry

- DetectionOrchestrator
- detection pipeline
- host trace adapter
- network event adapter
- persistence mapping
- 10 production detectors
- detector registration
- event-type scoping
- detector failure isolation
- API integration
- existing frontend integration
- tests

The architecture has already been independently reviewed.

The established conclusion was:

```
Phase 1: PASS WITH CORRECTIONS
Phase 2: READY WITH CORRECTIONS
```

Therefore:

> Build Phase 2 on top of Phase 1.

Do not perform an unnecessary architecture rewrite.

---

6. VERIFIED CURRENT PRODUCTION PIPELINE

The current production flow is approximately:

```
POST /api/analysis-runs/host-traces
        ↓
analysis.py
        ↓
summarize_host_trace()
        ↓
analyse_trace()
        ↓
CanonicalEvent[]
        ↓
DetectionContext
        ↓
DetectionOrchestrator
        ↓
DetectorRegistry
        ↓
enabled detectors
```

```
        ↓
DetectionResult[]
        ↓
persistence mapping
        ↓
Detection
        ↓
Alert
        ↓
API
        ↓
Frontend
```

Network analysis follows the corresponding network pipeline:

```
network telemetry
        ↓
CanonicalEvent[]
        ↓
DetectionContext
        ↓
DetectionOrchestrator
        ↓
network detectors
        ↓
DetectionResult[]
        ↓
persistence
```

Preserve this general flow.

---

7. CURRENT PHASE 1 DETECTORS

There are currently ten production detectors.

HOST

```
host.high_event_volume
host.excessive_sequences
host.suspicious_network_pattern
host.privilege_escalation
host.suspicious_file_operations
host.high_syscall_diversity
```

NETWORK

```
network.excessive_connection_attempts
network.port_scan_detected
network.high_connection_frequency
network.repeated_connection_attempts
```

All ten currently use hard-coded thresholds.

Phase 2 must migrate those thresholds into the new configuration mechanism while preserving their existing default values and behavior.

---

8. EXISTING THRESHOLDS — DO NOT CHANGE DEFAULT SEMANTICS

The established Phase 1 thresholds are:

```
500 events
50 sequences
3 network syscall types
2 sensitive file operations
30 unique system calls
10 unique destinations
5 source IPs per port
50 events per port
20 destinations/source
```

These values must remain the **default configuration values** unless the master Phase 2 document explicitly requires otherwise.

The objective is:

```
hard-coded
    ↓
configurable
```

NOT:

```
existing behavior
    ↓
arbitrarily changed behavior
```

---

9. CRITICAL TELEMETRY LIMITATION

There is an important repository-specific limitation that you MUST respect.

Current system-call canonical events are created with:

```
timestamp = None
```

The current system-call telemetry contains ordering information but does not contain actual timestamps.

Network events DO contain timestamps.

Therefore:

NETWORK

Temporal detection is supported:

```
rate
window
time-windowed sequence
```

SYSTEM CALLS

Current temporal detection cannot legitimately claim real-time windows.

System-call sequences can currently use:

```
event order
sequence_index
event_index
```

but cannot honestly claim:

```
within 5 seconds
within 30 seconds
within 1 minute
```

unless timestamps actually exist.

DO NOT fabricate timestamps.

DO NOT generate fake timestamps.

DO NOT infer fake timestamps merely to satisfy a detector.

If the Phase 2 master document permits an ordering-only fallback, implement it explicitly and document the limitation.

If the specification requires true temporal system-call detection, clearly isolate that limitation rather than silently implementing fake timing.

---

10. CRITICAL SCORE SEMANTICS

You MUST preserve the conceptual distinction between:

```
Heuristic Score
Confidence
Anomaly Score
Model Score
Threat Intelligence Confidence
Contextual Risk
Final Risk
Severity
```

These are NOT interchangeable.

In particular:

```
heuristic_score != confidence
```

The current database/API contains a historical naming problem:

```
Detection.confidence
```

currently stores:

```
result.scores.heuristic_score
```

The frontend also displays it as:

```
confidence%
```

You MUST follow the Phase 2 Master Builder document's explicit policy for resolving this.

Do NOT silently rename or break the API.

Do NOT silently pretend the heuristic score is a probabilistic confidence.

If compatibility requires temporarily preserving the field, make the semantic boundary explicit and document it.

---

11. PERSISTENCE FINDINGS THAT MUST BE ADDRESSED

Current `Detection` persistence does NOT contain:

```
detector_id
detector_version
```

Currently:

```
detector_id
```

is embedded indirectly in:

```
Alert.source
```

and:

```
detector_version
```

is not persisted.

Additionally:

```
DetectionResult.source_events
```

is not directly persisted.

You MUST follow the Phase 2 Master Builder specification regarding persistence.

If detector identity/version persistence is required, implement it properly using:

- SQLAlchemy model changes
- Alembic migration
- persistence mapping
- schemas

- API compatibility
- tests

Do NOT merely concatenate more information into an unrelated string field unless explicitly required.

---

12. EXISTING EVIDENCE STRUCTURE

The existing `Evidence` contract already contains much of the required foundation, including concepts such as:

```
rule
observed_values
matched_events
matched_fields
threshold
description
```

and:

```
to_dict()
```

Do NOT throw this away.

Extend it where required.

Phase 2 evidence should become richer while remaining compatible with:

```
Detection.evidence
```

which is currently persisted as JSON.

The frontend currently renders evidence as JSON.

Therefore, evidence must remain JSON serializable.

---

13. FRONTEND COMPATIBILITY

The frontend currently depends on fields such as:

```
confidence
severity
status
```

```
title
description
source_ip
evidence
```

Investigation currently renders:

```
detection.confidence
```

as a percentage.

Investigation also serializes:

```
detection.evidence
```

as JSON.

Alerts consume:

```
severity
status
title
description
source
created_at
```

Monitoring consumes detection/alert severity and network-related fields.

Therefore:

> Preserve API compatibility wherever reasonably possible.

If the Phase 2 specification requires a contract change, update:

1. backend schema
2. backend API
3. frontend types
4. frontend service layer
5. frontend components
6. tests

Do not leave the frontend using stale semantics.

---

14. PHASE 2 SCOPE

Phase 2 focuses on:

```
1. Configuration
2. Configurable deterministic rules
3. Rate/window detection
4. Sequence detection
5. IOC foundation
6. IOC matching/detection
7. Evidence enrichment
8. Persistence improvements required by Phase 2
9. Testing
10. Documentation
```

The exact requirements in the Master Builder document take precedence.

---

15. PHASE 2 EXCLUSIONS

Unless the Master Builder document explicitly changes the scope, DO NOT implement:

```
ML anomaly detection
Isolation Forest
LOF
One-Class SVM
Autoencoders
DeepSVDD
LIGHT-HIDS
behavioral baselines
concept drift
model training infrastructure
model registry
full threat-intelligence platform
full Sigma implementation
MITRE ATT&CK automation
full correlation engine
incident management platform
PCAP capture
live network capture
Zeek deployment
Suricata deployment
Kafka
Redis
Kubernetes
microservices
cloud infrastructure
```

```
SIEM
SOAR
automated response
```

Do not over-engineer Phase 2.

However, Phase 2 MUST be architecturally compatible with these future phases.

---

16. REQUIRED IMPLEMENTATION PHILOSOPHY

Use:

```
clean architecture
typed contracts
small modules
single responsibility
dependency inversion
testability
configuration over constants
provider abstractions
safe matching
bounded processing
explicit semantics
backward compatibility
```

Avoid:

```
god classes
god functions
hidden globals
magic constants
eval()
exec()
dynamic code execution
hardcoded IOC lists
fake telemetry
fake timestamps
fake hashes
fake threat intelligence
```

---

17. STEP 1 — REPOSITORY BASELINE BEFORE EDITING

Before modifying anything, inspect:

```
backend/app/detection/
backend/app/models/
backend/app/schemas/
backend/app/api/routes/
backend/app/services/
backend/tests/
frontend/src/
backend/alembic/
DOCS/
```

Inspect especially:

```
app/detection/contracts/core.py
app/detection/pipeline.py
app/detection/registry/registry.py
app/detection/registry/bootstrap.py
app/detection/orchestration/orchestrator.py
app/detection/scoring.py
app/detection/risk.py
app/detection/persistence/mapping.py
app/detection/adapters/host_trace.py
app/detection/detectors/host_detectors.py
app/detection/detectors/network_detectors.py
app/services/detection_engine.py
app/models/detection.py
app/models/alert.py
app/models/network_event.py
app/models/analysis_run.py
app/schemas/detections.py
app/schemas/analysis.py
app/schemas/network.py
app/api/routes/analysis.py
app/api/routes/detections.py
app/api/routes/alerts.py
app/api/routes/network.py
```

Also inspect:

```
frontend/src/features/alerts/
frontend/src/features/investigation/
frontend/src/features/monitoring/
```

```
frontend/src/features/network/
frontend/src/services/
frontend/src/types/
```

---

18. STEP 2 — ESTABLISH BASELINE TEST RESULT

Before implementation, run the existing backend test suite.

Record:

```
number passed
number failed
number skipped
number errors
```

Do not begin Phase 2 while unexplained pre-existing failures exist.

If the environment prevents tests from running, diagnose the environment first.

Do not modify unrelated application behavior simply to make tests pass.

---

19. STEP 3 — FIX PHASE 1 CORRECTIONS FIRST

Before adding major Phase 2 functionality, resolve the verified Phase 1 corrections.

19.1 Orchestrator lifecycle

Current problem:

```
pipeline.py
    ↓
module-level _orchestrator_instance
```

This caches the registry/orchestrator.

That prevents clean runtime detector configuration.

Implement the solution specified by the Master Builder document.

Requirements:

- avoid stale detector registry state

- ensure enabled/disabled configuration can be reflected
- avoid unnecessary global mutable state
- preserve testability
- preserve current pipeline behavior

Do not introduce an unnecessarily complex dependency injection framework.

---

20. LEGACY DETECTION ENGINE

Current file:

```
backend/app/services/detection_engine.py
```

This is legacy code.

It is not on the production detection path.

It is currently referenced by legacy tests.

The Master Builder document determines whether to:

- deprecate it
- migrate tests
- remove it
- retain a temporary compatibility facade

Do not delete it blindly.

Before removal, search the entire repository for imports/references.

Production code MUST NOT accidentally return to the legacy engine.

The new detection architecture must remain authoritative.

---

21. SCORING CONSOLIDATION

There are currently multiple heuristic-score implementations.

Known locations include:

```
app/detection/scoring.py
app/detection/detectors/host_detectors.py
```

```
app/detection/detectors/network_detectors.py
app/services/detection_engine.py
```

Create one authoritative scoring implementation.

Remove duplicated formulas from production detector code.

Detectors should report evidence/observations and let the scoring layer calculate the appropriate score according to the architecture.

Do NOT make scoring semantics more complicated than the specification requires.

---

22. DETECTOR CONFIGURATION SYSTEM

Implement the Phase 2 configuration architecture specified in the Master Builder document.

At minimum, support:

```
detector_id
enabled
threshold/configuration values
validation
defaults
typed configuration
```

Configuration must be:

```
safe
typed
validated
deterministic
testable
```

Do not allow arbitrary Python expressions.

Do not use:

```
eval(...)
```

or:

```
exec(...)
```

unless the specification explicitly requires a safe expression language — and even then, prefer a constrained parser.

---

23. DEFAULT CONFIGURATION

Existing Phase 1 behavior must remain the default.

For example:

```
host.high_event_volume
    threshold = 500

host.excessive_sequences
    threshold = 50

host.suspicious_network_pattern
    threshold = 3

host.suspicious_file_operations
    threshold = 2

host.high_syscall_diversity
    threshold = 30
```

and the network defaults must preserve their existing semantics.

Write tests proving:

```
default configuration
=
previous Phase 1 behavior
```

---

24. DETECTOR ENABLE/DISABLE

The configuration mechanism must allow detectors to be enabled/disabled according to the Phase 2 specification.

The registry/orchestrator must correctly respect:

```
enabled = true
enabled = false
```

Do not merely expose a configuration value that detectors ignore.

Write tests proving:

```
enabled detector → executes
disabled detector → does not execute
```

Also verify that changing configuration does not leave a stale cached registry.

---

25. CONFIGURATION VALIDATION

Invalid configuration must fail safely.

Examples:

```
negative threshold
invalid type
unknown detector
missing required parameter
invalid window duration
invalid sequence definition
invalid IOC
```

must be handled according to the specification.

Do not allow malformed configuration to crash unrelated detectors.

---

26. RATE/WINDOW FRAMEWORK

Build a reusable temporal framework.

The framework should conceptually support:

```
fixed windows
sliding windows
event rates
counts within windows
grouping
minimum/maximum thresholds
```

Do not implement unnecessary variants if the Master Builder document only requires a subset.

The framework should be reusable by future detectors.

---

27. TIMESTAMP SAFETY

The temporal framework MUST explicitly handle:

```
timestamp = None
```

Do not silently treat missing timestamps as:

```
now()
```

Do not fabricate time.

Possible valid behavior:

```
skip temporal detector
```

or:

```
use ordering-only logic
```

depending on the detector/specification.

The behavior must be explicit and tested.

---

28. FIRST RATE/WINDOW DETECTOR

Implement the rate/window detector specified by the Master Builder.

Prefer a detector that can operate on the current telemetry.

Because network events have real timestamps, network-scoped rate detection is currently the safest temporal implementation.

The detector must:

- consume CanonicalEvent
- use configuration
- produce DetectionResult
- produce Evidence
- preserve detector identity/version
- avoid direct DB access
- remain independently testable
- avoid quadratic behavior

29. RATE DETECTION PERFORMANCE

Do not implement naive:

```
for every event:
    scan all other events
```

if a sliding-window or bounded-window implementation can avoid it.

Prefer appropriate structures such as:

```
deque
grouped counters
sorted event streams
```

depending on the specification.

Complexity should be considered explicitly.

---

30. SEQUENCE DETECTION FRAMEWORK

Implement a reusable predicate sequence matcher.

The conceptual input should support patterns such as:

```
[A, B]
```

and:

```
[A, B, C]
```

meaning:

```
A then B
```

or:

```
A then B then C
```

The matcher must preserve ordering.

---

31. SEQUENCE PREDICATES

The predicate system must be safe and constrained.

Do not allow arbitrary Python code.

Predicates should be represented through typed/configurable fields.

Possible predicate attributes include:

```
event_type
syscall
process_name
source_ip
destination_ip
protocol
file_path
attributes
```

Use only fields actually available in the canonical event.

---

32. SEQUENCE MATCH RESULT

A sequence match should preserve enough information to explain what happened.

Where available, include:

```
sequence definition
matched events
matched fields
event order
start timestamp
end timestamp
span
```

If timestamps are unavailable:

```
timestamp = null
```

must remain honest.

Do not manufacture a duration.

---

33. SYSTEM-CALL SEQUENCE LIMITATION

Current syscall telemetry contains ordering but no timestamps.

Therefore:

```
A → B → C
```

is legitimate.

But:

```
A → B → C within 10 seconds
```

is NOT currently legitimate for syscall events.

Implement the supported semantics and explicitly document the limitation.

---

34. SEQUENCE DETECTOR

Implement the sequence detector specified in the master document.

It must:

- implement Detector
- be registered
- have stable detector ID
- have detector version
- be independently configurable
- use the sequence framework
- produce DetectionResult
- produce structured Evidence
- preserve matched-event information
- avoid DB coupling
- be failure-isolated by the orchestrator

---

35. IOC FOUNDATION

Implement a clean IOC abstraction.

Potential IOC types include:

```
IP
domain
URL
file hash
process
file path
```

BUT:

> Only match against telemetry fields that are actually populated.

Current telemetry genuinely supports network IP information.

Current telemetry does NOT reliably populate:

```
domain
url
file_hash
```

Do not fabricate IOC data.

---

36. IOC NORMALIZATION

Implement safe normalization.

For example:

```
IP addresses
domains
URLs
hashes
```

should have deterministic normalization rules.

Be careful not to alter the semantic identity of an IOC.

Do not normalize two different values into one incorrectly.

---

37. IOC PROVIDER

Use an abstraction such as:

```
IOCProvider
```

or the equivalent required by the master document.

The detector should not be tightly coupled to:

```
hardcoded global IOC lists
```

Prefer:

```
provider
    ↓
normalized IOC set
    ↓
matcher
```

This will support future external threat intelligence without rewriting the detector.

---

38. IOC MATCHER

The matcher should use efficient lookup.

Prefer:

```
set
dict
indexed structure
```

rather than:

```
for event:
    for every IOC:
```

where practical.

The matcher must return explainable match information.

---

39. IOC DETECTOR

Implement the IOC detector according to the Master Builder specification.

It must:

- use actual telemetry
- not fabricate values
- identify matching IOC type
- identify the matched value safely
- provide evidence
- include detector ID/version
- integrate with the orchestrator
- remain independently testable

---

40. REAL TELEMETRY ONLY

This rule is absolute:

> NEVER manufacture security evidence.

Do not create:

```
fake malicious IP
fake hash
fake URL
fake domain
fake process
fake timestamp
fake attack
```

unless the value is explicitly part of a test fixture.

Test fixtures are allowed.

Production detection logic must operate on actual supplied telemetry.

---

41. EVIDENCE ENRICHMENT

Improve the evidence model as specified.

Evidence should answer:

```
What detector matched?
What rule/configuration matched?
What was observed?
What threshold was crossed?
Which events matched?
```

```
Which fields matched?
What sequence matched?
What IOC matched?
What time window was evaluated?
```

Do not put unnecessary raw telemetry into every detection.

Evidence must remain bounded.

---

42. EVIDENCE SIZE

Avoid unbounded evidence.

A malicious or pathological trace must not be able to create enormous database JSON records simply because a detector matched thousands of events.

Implement sensible limits specified by the master document.

If truncation occurs, evidence should indicate that it was truncated.

---

43. DETECTOR ID/VERSION PERSISTENCE

If required by the Master Builder document, extend persistence so every detection can answer:

```
which detector generated this?
which detector version generated this?
```

Do this structurally.

Potential implementation:

```
Detection.detector_id
Detection.detector_version
```

with:

```
Alembic migration
```

and corresponding schema/API changes.

Do not rely solely on:

```
Alert.source
```

for machine-readable identity.

---

44. SOURCE EVENT PERSISTENCE

If required by the specification, provide an appropriate representation of source/matched events.

Do not blindly serialize massive CanonicalEvent objects.

Prefer a bounded, stable evidence representation.

Maintain:

```
source event identity
event index
matched fields
relevant event data
```

as appropriate.

---

45. DATABASE MIGRATIONS

If SQLAlchemy models change:

YOU MUST create a proper Alembic migration.

Do not simply modify the model.

Do not rely on:

```
Base.metadata.create_all(...)
```

to represent production schema migration.

Migration must be:

- deterministic
- reversible where practical
- tested
- compatible with existing development data

---

46. API COMPATIBILITY

Before changing an API field:

Search all usages.

Search:

```
backend
frontend
tests
documentation
scripts
```

For every changed field, determine:

```
producer
consumer
storage
serialization
display
tests
```

Do not break one layer accidentally.

---

47. CONFIDENCE FIELD

Pay special attention to:

```
Detection.confidence
```

because it currently contains:

```
heuristic_score
```

and the frontend displays:

```
confidence%
```

The Master Builder document's score semantics must be implemented honestly.

If backward compatibility requires retaining `confidence`, consider whether it should temporarily remain as a compatibility field while introducing a semantically correct field.

Do not expose false probability claims.

---

48. FRONTEND CHANGES

Do not modify the frontend unnecessarily.

Phase 2 is primarily backend/detection-engine work.

However, if the API contract changes:

- update frontend types
- update services
- update components
- update displayed labels
- ensure evidence renders correctly

Do not leave:

```
heuristic score
```

labeled:

```
confidence
```

if the backend has changed its semantics.

---

49. TESTING STRATEGY

Testing is a first-class deliverable.

For every new component:

```
unit tests
integration tests
regression tests
```

where appropriate.

---

50. CONFIGURATION TESTS

Test:

```
default configuration
custom threshold
invalid threshold
disabled detector
enabled detector
unknown detector
configuration isolation
runtime configuration refresh
```

---

51. RATE/WINDOW TESTS

Test:

```
events inside window
events outside window
threshold reached
threshold not reached
boundary timestamps
missing timestamps
multiple groups
empty input
large input
```

Do not test only the happy path.

---

52. SEQUENCE TESTS

Test:

```
A → B
A → B → C
wrong order
partial match
multiple matches
overlapping matches
empty sequence
single-element sequence
timestamped events
```

```
untimestamped syscall events
optional window
window exceeded
```

---

53. IOC TESTS

Test:

```
matching IP
non-matching IP
normalized IP
multiple IOCs
duplicate IOC
invalid IOC
empty IOC provider
unsupported telemetry field
```

Also explicitly prove:

```
no fabricated IOC matches
```

---

54. EVIDENCE TESTS

Verify that detection evidence contains the information needed to explain the detection.

Test:

```
rule
threshold
observed value
matched event
matched field
sequence evidence
IOC evidence
window evidence
truncation behavior
JSON serialization
```

---

55. PERSISTENCE TESTS

If persistence is changed, test:

```
detector_id persists
detector_version persists
evidence persists
API returns expected fields
existing detections remain readable
migration succeeds
```

---

56. REGRESSION TEST

The entire Phase 1 suite must continue passing.

The original 196-test baseline must not regress unless tests have been intentionally migrated because behavior/contracts legitimately changed.

Any changed test must have a documented reason.

Do not simply delete failing tests.

---

57. TEST THE REAL PIPELINE

Unit tests are insufficient.

Perform an actual end-to-end test:

```
sample-data/example-host-trace.json
        ↓
upload
        ↓
analysis
        ↓
canonical events
        ↓
detectors
        ↓
DetectionResult
        ↓
database
        ↓
API
```

```
        ↓
frontend-compatible response
```

Verify that existing detection behavior remains functional.

---

58. NETWORK REGRESSION

Test existing network functionality.

Ensure Phase 2 does not break:

```
network summary
network events
IP investigation
ports
protocols
top talkers
timeline
network detections
```

---

59. LEGACY TEST MIGRATION

Because:

```
tests/test_network.py
```

currently references the legacy:

```
services.detection_engine
```

you MUST inspect those tests before removing/deprecating the legacy module.

The goal is:

```
production tests
    ↓
new architecture
```

not:

```
production
    ↓
new architecture

tests
    ↓
old architecture
```

unless the legacy compatibility layer is intentionally retained.

---

60. SECURITY REQUIREMENTS

Review all new configuration and matching logic for:

Injection

No:

```
eval
exec
shell execution
unsafe dynamic imports
```

Resource exhaustion

Bound:

```
IOC count
sequence length
window size
evidence size
configuration size
event matching work
```

Data validation

Validate:

```
types
ranges
formats
lengths
```

---

61. PERFORMANCE REQUIREMENTS

Avoid unnecessary:

```
O(n²)
```

operations.

Particularly inspect:

```
sequence matching
IOC matching
rate windows
evidence construction
```

Use efficient data structures.

Do not optimize prematurely, but do not knowingly implement pathological algorithms.

---

62. DETECTOR FAILURE ISOLATION

A failure in one new detector must not prevent unrelated detectors from running.

For example:

```
IOC detector fails
```

must not prevent:

```
host.high_event_volume
```

from executing.

Preserve the existing orchestrator failure-isolation behavior.

---

63. DETECTOR REGISTRATION

Every new detector must:

- have stable ID
- have version

- have name
- have description
- have category
- have scope
- have enabled state
- satisfy the Detector contract
- be registered through the proper registry mechanism

Do not manually invoke new detectors from routes.

---

64. NO ROUTE-LEVEL DETECTION LOGIC

Do not put detection logic inside:

```
analysis.py
network.py
detections.py
alerts.py
```

Routes should orchestrate API behavior.

Detection belongs in:

```
app/detection/
```

---

65. NO DATABASE ACCESS FROM DETECTORS

Detectors must not directly import:

```
SQLAlchemy session
ORM models
database engine
```

They should return:

```
DetectionResult
```

Persistence remains separate.

---

66. CONFIGURATION DEPENDENCY DIRECTION

Prefer:

```
configuration
    ↓
DetectionContext
    ↓
Detector
```

rather than:

```
Detector
    ↓
global config singleton
```

Keep configuration injectable and testable.

---

67. FUTURE PHASE COMPATIBILITY

Phase 2 must make future phases easier.

The architecture should allow future:

```
statistical detectors
behavioral baselines
Isolation Forest
LOF
One-Class SVM
LIGHT-HIDS
DeepSVDD
```

to plug into the same detection architecture.

Do NOT implement those algorithms now.

---

68. NO FAKE LIGHT-HIDS

Do not add a fake:

```
LIGHT-HIDS score
```

Do not rename heuristic score to:

```
LIGHT-HIDS anomaly score
```

Do not create a placeholder ML detector pretending to be implemented.

If status documentation mentions future ML, clearly label it as:

```
planned
not implemented
```

---

69. DOCUMENTATION

After implementation, update/create the Phase 2 documentation required by the Master Builder document.

At minimum document:

```
architecture
configuration
detector configuration
rate/window semantics
sequence semantics
IOC architecture
evidence
persistence
API changes
timestamp limitations
score semantics
testing
future Phase 3 extension points
```

Do not write documentation that claims features that are not actually implemented.

---

70. CODE QUALITY

Before finalizing:

Inspect all changed files.

Look for:

```
unused imports
dead code
duplicated functions
magic constants
incorrect types
incorrect Optional handling
unbounded loops
unbounded data
incorrect timezone behavior
misleading names
debug prints
temporary scripts
TODOs accidentally left behind
```

Remove temporary implementation artifacts.

---

71. DO NOT TOUCH UNRELATED SYSTEMS

Avoid modifying:

```
authentication
RBAC
audit architecture
unrelated user management
unrelated frontend pages
unrelated network analytics
```

unless required by a documented compatibility issue.

If you discover unrelated bugs:

- do not silently expand scope
- document them
- only fix if required for Phase 2 correctness/security

---

72. ENVIRONMENT SAFETY

The development environment uses:

```
Python 3.14
FastAPI
```

```
SQLAlchemy 2.0
pytest
SQLite tests
PostgreSQL configured for development
```

Tests currently use in-memory SQLite.

Do not break this arrangement.

Do not require PostgreSQL to be running merely to execute unit tests unless the specification explicitly requires an integration test.

---

73. MIGRATION SAFETY

Before creating migrations:

inspect current Alembic state.

Do not create duplicate or conflicting migrations.

After creating migration:

test upgrade/downgrade where practical.

---

74. GIT PREPARATION

After all implementation and testing is complete:

Inspect:

```
C:\Users\dhruv\SentinalAI
```

for:

```
.env
credentials
passwords
JWT secrets
database files
logs containing secrets
temporary files
Python caches
node_modules
```

```
build artifacts
IDE metadata
```

Do NOT commit secrets.

---

75. CREATE .gitignore IF REQUIRED

Because this directory currently has no Git repository, determine whether a proper `.gitignore` exists.

If not, create one appropriate for:

```
Python
FastAPI
pytest
React
Vite
Node
pnpm
VS Code
Windows
SQLite
environment files
```

At minimum ensure files such as:

```
.env
*.db
__pycache__/
.pytest_cache/
node_modules/
dist/
build/
```

and other generated/secrets files are ignored where appropriate.

Do not blindly ignore source files.

---

76. GIT INITIALIZATION

If `.git` does not exist:

```
git init
```

Use the repository root:

```
C:\Users\dhruv\SentinalAI
```

Do not initialize separate Git repositories inside:

```
backend/
frontend/
```

There should be ONE root repository.

---

77. CHECK GIT IDENTITY

Before committing, inspect:

```
git config user.name
git config user.email
```

If configured, use the existing identity.

Do not invent a Git identity.

If not configured, report that configuration is required rather than fabricating one.

---

78. GITHUB REMOTE

The user wants the completed project pushed to GitHub.

First inspect whether a remote already exists:

```
git remote -v
```

If a remote exists:

- inspect it
- do not overwrite it blindly
- determine whether it points to the intended SentinelAI repository

If no remote exists:

You MUST determine the intended GitHub repository before pushing.

Do not invent a repository URL.

If GitHub CLI is installed and authenticated, you may inspect available repositories.

If authentication is unavailable or repository identity cannot be determined, STOP BEFORE PUSHING and report exactly what is needed.

Do not fabricate credentials or URLs.

---

79. GITHUB REPOSITORY VISIBILITY

Do not change repository visibility unless explicitly instructed.

If the intended repository already exists and is private:

```
keep it private
```

unless the user explicitly asks otherwise.

---

80. GIT STATUS AUDIT

Before committing:

```
git status
```

Inspect every untracked/modified file.

Do not blindly run:

```
git add .
```

without first checking what will be included.

Especially verify that:

```
.env
database files
credentials
tokens
```

```
password reset data
local logs
```

are excluded.

---

81. FIRST COMMIT

The initial repository commit should represent the verified complete SentinelAI project state after Phase 2 implementation.

Suggested commit message:

```
feat: implement SentinelAI analysis engine phase 2
```

Use a different concise message only if the repository state makes another message more accurate.

---

82. PUSH TO GITHUB

After:

```
tests pass
security audit passes
git status reviewed
remote verified
commit created
```

push to the intended remote.

If the branch is new, configure the upstream appropriately.

Do not force-push.

Do not use:

```
git push --force
```

unless the user explicitly authorizes it.

---

83. IMPORTANT GITHUB SAFETY RULE

If authentication fails:

DO NOT:

- ask for passwords in code
- store tokens in files
- hardcode credentials
- disable security controls
- fabricate a successful push

Instead report:

```
implementation complete
commit complete
push blocked by GitHub authentication
```

and provide the exact next command/action required.

---

84. FINAL REPOSITORY VERIFICATION

After pushing, verify:

```
git status
```

Expected:

```
working tree clean
```

Then verify the commit:

```
git log -1
```

If possible, verify the remote branch contains the commit.

---

85. FINAL TEST SUITE

Run the complete test suite again AFTER all implementation and migrations.

Report:

```
total tests
passed
failed
skipped
errors
```

Do not claim success without actually running tests.

---

86. FINAL FUNCTIONAL VERIFICATION

Perform at least:

Host trace

```
sample host trace
    ↓
analysis
    ↓
detections
    ↓
alerts
```

Network trace

Verify:

```
network analysis
network detections
network persistence
```

Phase 2

Verify at least:

```
configuration
rate/window detector
sequence detector
IOC detector
evidence enrichment
```

according to what the Master Builder document requires.

---

87. FINAL ARCHITECTURE AUDIT

After implementation, inspect the resulting architecture and verify:

```
routes
    ↓
pipeline
    ↓
orchestrator
    ↓
registry
    ↓
detectors
    ↓
DetectionResult
    ↓
persistence
```

There must be no accidental path:

```
route
    ↓
legacy detection_engine.py
```

---

88. FINAL SEARCH FOR LEGACY ENGINE USAGE

Search the repository for:

```
services.detection_engine
detection_engine
analyse_network_events
_compute_confidence
_compute_heuristic_score
```

Determine which references remain.

There must be no accidental production usage of the legacy engine.

---

89. FINAL SEARCH FOR BAD PRACTICES

Search for:

```
eval(
exec(
TODO
FIXME
print(
hardcoded IOC
fake timestamp
fake hash
```

Review each result.

Do not mechanically delete legitimate documentation/test references.

---

90. FINAL SCORE SEMANTICS AUDIT

Search for:

```
confidence
heuristic_score
anomaly_score
risk_score
severity
```

Verify that they are not accidentally conflated.

Particularly verify:

```
DetectionResponse
Detection model
DetectionResult
DetectionScores
frontend display
```

---

91. FINAL TIMESTAMP AUDIT

Search for CanonicalEvent creation.

Verify that:

```
system-call timestamp
```

is not fabricated.

Verify temporal detectors handle missing timestamps correctly.

---

92. FINAL EVIDENCE AUDIT

Inspect real DetectionResult output.

Confirm evidence is:

```
structured
JSON serializable
bounded
meaningful
accurate
```

Verify it does not contain fake security evidence.

---

93. FINAL CONFIGURATION AUDIT

Verify:

```
defaults
custom configuration
disabled detector
enabled detector
invalid configuration
```

all behave according to specification.

---

94. FINAL API AUDIT

Verify existing endpoints still work.

At minimum inspect:

```
analysis runs
detections
alerts
network
```

```
monitoring
authentication
```

Do not introduce accidental API regressions.

---

95. FINAL FRONTEND AUDIT

If backend response structures changed:

Verify the frontend still:

```
loads
builds
renders
displays detections
displays alerts
displays evidence
displays severity
```

Run the frontend build if available.

Do not claim frontend success without actually checking it.

---

96. FINAL DOCUMENTATION AUDIT

Documentation must accurately reflect:

```
implemented Phase 1
implemented Phase 2
known telemetry limitations
known future work
score semantics
configuration behavior
detector list
test status
```

Never document future functionality as implemented.

---

97. DO NOT STOP AFTER FIRST SUCCESS

A coding task is NOT complete merely because:

```
new files compile
```

You must verify:

```
architecture
tests
database
API
frontend compatibility
security
performance
Git
GitHub
```

---

98. IMPLEMENTATION ORDER

Follow this general order unless the Master Builder document gives a more precise order:

```
1. Read master document
2. Inspect repository
3. Run baseline tests
4. Fix orchestrator lifecycle
5. Deprecate/migrate legacy engine
6. Consolidate scoring
7. Establish score semantics
8. Establish detector configuration
9. Migrate existing detector thresholds
10. Implement temporal primitives
11. Implement rate detector
12. Implement sequence matcher
13. Implement sequence detector
14. Implement IOC contracts
15. Implement IOC provider
16. Implement IOC normalizer
17. Implement IOC matcher
18. Implement IOC detector
19. Enrich evidence
20. Implement required persistence changes
21. Add migrations
22. Update schemas/API
23. Update frontend only where required
24. Add unit tests
25. Add integration tests
26. Run complete suite
27. Fix regressions
28. Perform security audit
```

```
29. Perform performance review
30. Update documentation
31. Final repository audit
32. Initialize Git
33. Create .gitignore if necessary
34. Inspect Git status
35. Verify remote
36. Commit
37. Push
38. Verify clean working tree
39. Produce final report
```

---

99. WHEN YOU ENCOUNTER AN AMBIGUITY

Do not silently make a major architectural decision.

Use this hierarchy:

```
Phase 2 Master Builder document
        ↓
verified repository reality
        ↓
Phase 1 architecture
        ↓
existing API compatibility
        ↓
safe minimal implementation
```

If a decision is still genuinely ambiguous:

- choose the smallest safe option if implementation can continue
- document the decision
- do not invent requirements
- do not expand scope unnecessarily

---

100. WHEN A REQUIREMENT CONFLICTS WITH CURRENT TELEMETRY

Do not fake the telemetry.

Example:

```
Requirement:
rate detection within time window

Reality:
system-call timestamps unavailable
```

Correct response:

```
implement temporal detection only where timestamps exist
OR
implement ordering-only sequence semantics where explicitly allowed
AND
document limitation
```

Incorrect response:

```
invent timestamps
```

---

101. WHEN A TEST FAILS

Classify the failure:

```
implementation bug
test expectation outdated
environment problem
pre-existing failure
migration problem
API compatibility problem
```

Fix the correct layer.

Do not weaken assertions merely to obtain green tests.

---

102. WHEN A MIGRATION FAILS

Inspect:

```
current database
Alembic revision
metadata
```

```
model state
migration dependency
```

Do not delete migration history casually.

Do not use:

```
drop database
```

or destructive resets against a real development database unless explicitly authorized.

---

103. WHEN GITHUB PUSH FAILS

Do not claim success.

Report:

```
commit SHA
remote
branch
exact authentication/push failure
```

and what the user must do next.

---

104. FINAL REPORT FORMAT

At the end, provide a detailed implementation report.

Include:

A. Master Prompt

```
Located:
[path]
Read completely:
YES
```

B. Implementation Summary

List all Phase 2 components implemented.

C. Phase 1 Corrections

List:

```
orchestrator lifecycle
legacy engine
scoring
risk boundary
```

and status.

D. Configuration

Explain:

```
architecture
defaults
validation
runtime behavior
```

E. Rate/Window

Explain:

```
framework
detector
timestamp limitations
tests
```

F. Sequence Detection

Explain:

```
matcher
predicates
ordering
time windows
limitations
```

G. IOC

Explain:

```
provider
normalizer
matcher
detector
supported telemetry
unsupported telemetry
```

H. Evidence

Explain what was added.

I. Persistence

Explain:

```
model changes
migration
detector identity
version
source event handling
```

J. API

Explain any changes.

K. Frontend

Explain any changes.

L. Tests

Provide actual:

```
passed
failed
skipped
errors
```

M. Security

List checks performed.

N. Performance

List important complexity/bounding decisions.

O. Documentation

List documentation added/updated.

P. Git

Provide:

```
Git initialized: YES/NO
branch:
commit:
commit message:
working tree:
```

Q. GitHub

Provide:

```
remote configured:
repository:
branch pushed:
push successful:
```

Do NOT expose credentials or tokens.

R. Remaining Limitations

Be completely honest.

S. Phase 3 Readiness

Explain whether the Phase 2 architecture is ready for:

```
statistical detection
behavioral baselines
ML
LIGHT-HIDS
```

without claiming those features are implemented.

---

105. ABSOLUTE RULES

These rules override convenience:

RULE 1

Do not fabricate telemetry.

RULE 2

Do not fabricate test results.

RULE 3

Do not fabricate GitHub push success.

RULE 4

Do not expose credentials.

RULE 5

Do not use fake timestamps.

RULE 6

Do not use fake hashes.

RULE 7

Do not use fake threat intelligence.

RULE 8

Do not use `eval()` or `exec()` for detection configuration.

RULE 9

Do not bypass the DetectionOrchestrator.

RULE 10

Do not put detection logic into API routes.

RULE 11

Do not couple detectors directly to SQLAlchemy.

RULE 12

Do not redesign Phase 1 unnecessarily.

RULE 13

Do not implement Phase 3 early.

RULE 14

Do not silently break frontend contracts.

RULE 15

Do not silently change existing default thresholds.

RULE 16

Do not silently conflate heuristic score with confidence.

RULE 17

Do not silently turn transitional risk scoring into a fake production risk engine.

RULE 18

Do not delete legacy code without checking every reference.

RULE 19

Do not commit `.env`, secrets, credentials, or sensitive local data.

RULE 20

Do not force-push to GitHub.

---

106. DEFINITION OF DONE

Phase 2 is complete only when ALL applicable conditions below are satisfied:

```
[ ] Master Builder document completely read
[ ] Repository inspected
[ ] Baseline tests recorded
[ ] Phase 1 preserved
[ ] Orchestrator lifecycle corrected
[ ] Legacy engine handled correctly
[ ] Scoring consolidated
[ ] Score semantics corrected/documented
```

```
[ ] Detector configuration implemented
[ ] Existing thresholds configurable
[ ] Existing defaults preserved
[ ] Rate/window framework implemented
[ ] Rate detector implemented
[ ] Timestamp limitation handled honestly
[ ] Sequence matcher implemented
[ ] Sequence detector implemented
[ ] IOC foundation implemented
[ ] IOC normalization implemented
[ ] IOC matcher implemented
[ ] IOC detector implemented
[ ] No fabricated IOC data
[ ] Evidence enriched
[ ] Evidence bounded
[ ] Required persistence changes implemented
[ ] Alembic migrations created if needed
[ ] API contracts verified
[ ] Frontend compatibility verified
[ ] Unit tests added
[ ] Integration tests added
[ ] Regression suite passes
[ ] Security review completed
[ ] Performance review completed
[ ] Documentation updated
[ ] No accidental legacy production path
[ ] Git repository initialized
[ ] .gitignore verified
[ ] Secrets excluded
[ ] Git status reviewed
[ ] Git commit created
[ ] GitHub remote verified
[ ] GitHub push completed OR blocker reported honestly
[ ] Final working tree verified
[ ] Final implementation report produced
```

---

107. MOST IMPORTANT PRINCIPLE

The goal is NOT to produce the largest amount of code.

The goal is to produce a **real, maintainable, testable Phase 2 detection engine** that is:

```
modular
accurate
explainable
configurable
safe
```

```
performant
backward-compatible
extensible
```

The resulting architecture must make the next phases easier rather than forcing another detection-engine rewrite.

---

108. START NOW

Begin with:

```
1. Locate the Phase 2 Master Builder Word document.
2. Read it completely.
3. Inspect the repository.
4. Run the baseline tests.
5. Compare the document requirements against the actual repository.
6. Implement Phase 2 systematically.
7. Test continuously.
8. Perform the final audit.
9. Initialize Git if necessary.
10. Commit the complete project.
11. Push to the intended GitHub repository.
12. Report exactly what happened.
```

Do NOT skip directly to coding before reading the master document.

Do NOT stop after implementing only the first feature.

Do NOT declare completion until the full Phase 2 scope has been implemented and verified.

Begin.